CS 4375 HW3: Neural Network Report

Shivesh Gupta

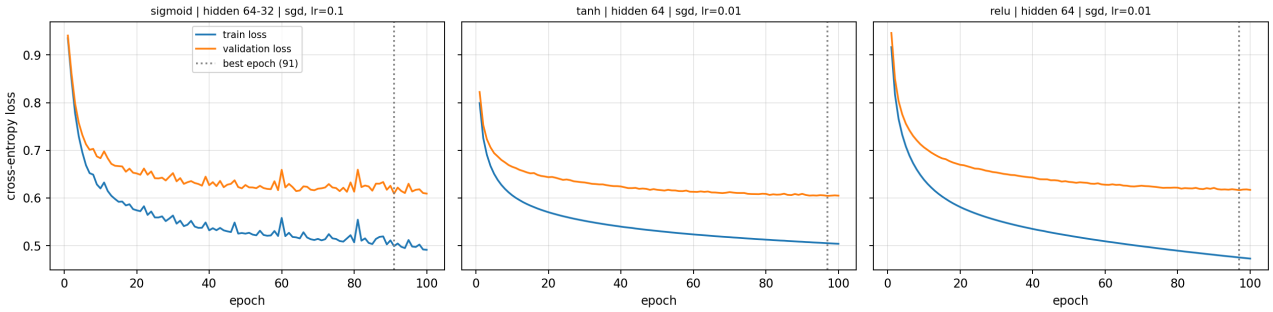
October 2026

Setup

I used the UCI *Predict Students' Dropout and Academic Success* dataset (`https://archive.ics.uci.edu/dataset/697`). It has 4,424 students, 36 features and 3 classes: Graduate (49.9%), Dropout (32.1%) and Enrolled (17.9%). I split it 70/15/15 with stratification (3096 / 664 / 664) and used seed 42. I one-hot encoded the 10 categorical columns and standardized the rest, which gives 234 inputs. The output layer is softmax with cross-entropy. The classes are not balanced, so I also report macro-F1, and I used validation macro-F1 to pick the best configuration.

I ran 24 configurations: 3 activations × 2 architectures (64 and 64–32) × SGD (lr 0.1, 0.01) or Adam (lr 0.01, 0.001). Training stops early once the validation loss has not improved for 15 epochs (100 epochs at most), and I keep the best weights. The full table is in `results.pdf`.

Activation	Hidden	Optimizer (lr)	Train acc	Val acc	Val F1	Test acc	Test F1
Sigmoid	64–32	SGD (0.1)	80.17	77.11	70.99	79.67	73.09
Tanh	64	SGD (0.01)	79.43	77.41	69.95	79.07	71.46
ReLU	64	SGD (0.01)	80.81	77.56	70.57	78.61	70.43
Logistic regression baseline					77.26	78.16	71.35



(a) Which activation worked best, and why?

By my rule (highest validation macro-F1), sigmoid won. It also had the best test accuracy, 79.67%. But the three activations are very close. Test accuracy ranges from 78.6% to 79.7%, and with only 664 test students one standard error is about 1.6 points, so I can't say any of them is really better. Averaged over all 24 runs, validation accuracy was 76.3% for sigmoid, 76.6% for tanh and 76.2% for ReLU.

The learning curves do show what Q1.3 predicts. ReLU gets the lowest training loss (about 0.47 at epoch 100) because its derivative is 1 for positive inputs, so the gradient doesn't shrink. Sigmoid is the slowest. Its best runs needed the biggest learning rate (0.1), and on average its best epoch was 51, compared with 23–24 for tanh and ReLU. Its curve is also noisier. This matches the fact that $\sigma'(z) \leq 0.25$. With only one or two hidden layers the vanishing-gradient problem is mild, so it slows sigmoid down but doesn't break it. All three validation losses level off around 0.60–0.62, so the limit seems to come from the data and not from the activation.

(b) Optimizer versus plain SGD

I compared SGD with Adam. Adam converges much faster: its lowest validation loss came after 8.6 epochs on average, compared with 56.8 for SGD, which is about 6.6 times fewer epochs. But it did not give a better final result. Across all runs, the average validation accuracy was 76.3% for Adam and 76.4% for SGD, and the average validation macro-F1 was 68.6% and 67.9%. In fact, the best configuration for every activation ended up using SGD, although only by small margins. Adam is also sensitive to the learning rate: with lr 0.01 the 64-unit ReLU network only reached 74.7% validation accuracy, because the steps were too big. So on this dataset Adam saves time but not accuracy.

(c) Overfitting

Yes, and it is clearest with Adam. For ReLU (64 units, Adam, lr 0.001), training accuracy was 83.0% and validation accuracy was 76.2%, a gap of 6.8 points. The validation loss was lowest at epoch 10 and never improved after that, so early stopping ended the run at epoch 25. Other Adam runs look the same (their best epochs were between 1 and 27). The 64-unit SGD runs with lr 0.01 do not overfit: their validation loss is still slowly falling at epoch 97, so they stop because of the epoch limit and not because they converged.

Here is what I would do: keep early stopping, add L2 weight decay (it is implemented as `--l2`, but I did not tune it), use dropout, and cut down the inputs. Many of the 234 features come from rare category values, such as the parents' occupations, which a network can memorize. A smaller hidden layer or more data would also help.

(d) Comparison with logistic regression

Logistic regression on the same split gets 78.16% test accuracy and 71.35% macro-F1. The best network gets 79.67% and 73.09%. The other two networks get 79.07% and 78.61% accuracy, and the ReLU network has a lower macro-F1 (70.43%). So the differences are about one point or less, which is within the noise. To me this means the link between the features and the outcome is mostly linear, and a nonlinear network adds little. The hard class is *Enrolled*: for the baseline its F1 is only 0.47, compared with 0.81 for Dropout and 0.86 for Graduate. Students who are still enrolled sit between the other two groups and are hard to tell apart with these features. That is why macro-F1 is so much lower than accuracy.

Assumptions

- The 10 integer-coded columns (marital status, application mode, course, attendance, qualifications, nationality, and the parents' qualifications and occupations) are categories, so I one-hot encoded them. All other columns are numeric, including the 0/1 columns.
- The output layer always uses Xavier initialization because it feeds a softmax. Hidden layers use He (ReLU) or Xavier (sigmoid, tanh).
- Each configuration was run once (seed 42), and training was capped at 100 epochs.
- I used the test set only for the best configuration of each activation and for the baseline.

References

[1] Realinho et al. (2021). *Predict Students' Dropout and Academic Success* (dataset). UCI Machine Learning Repository. https://doi.org/10.24432/C5MC89

[2] Ruder, S. (2016). *An overview of gradient descent optimization algorithms.* https://arxiv.org/abs/1609.04747 (momentum, RMSProp, Adam)

[3] Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*, Ch. 8. MIT Press. https://www.deeplearningbook.org (optimization)

AI assistance: an AI assistant was used to help draft the theory part (Part 1) and to polish the wording of the report and results documents.